Projekt 1: „MCU Command Console” – sterowanie i diagnostyka przez UART

Cel: napisać firmware w języku C na STM32G491RE, który wykonuje następującą funkcjonalność **bez blokowania pętli głównej**.

---

1) Architektura

1.1. Pętla główna (zakaz blokowania)

- `main()` robi tylko:
    - Zarządza cyklicznymi zadaniami
        - Co 500ms zmiana stanu diody LED (tą częstotliwość będziemy potem zmieniać)
        - Co 2200ms szybkie 2 krotne zamruganie dioda LED
        - Co 1000ms wysyłanie Logów z wszystkimi aktualnymi parametrami

Zasada: żadnych `HAL_Delay()` i podobnych w logice. Żadne zadanie nie może blokować pozostałych. Do zarządzania zadaniami używam Timera.

1.2. UART jako konsola (z DMA)

Należy zaplanować język komunikacji z urządzeniem. Przygotować dokumentację jak wygląda ramka komunikatów, np. Komendy AT “AT+CSQ”. Rozpisać sobie kilka rodzajów błędów jak np. złe rozpoczęcie komunikatu, błędna druga część komendy, nieznana komenda, niekompletna komenda itp. Dla każdego takiego błędu wygeneruj indywidualną sekwencje mrugnięć diody LED.

- Odbiór UART:
    - DMA do bufora kołowego
    - parser linii: komendy kończą się `\r\n`
- Nadawanie UART:
    - DMA TX (żeby logi nie blokowały)

1.3. Logi + diagnostyka

- Makra typu:
    - `LOGI("...")`, `LOGW`, `LOGE`
- Opcjonalnie „kanał pomiarowy” GPIO: szybkie toggle do analizatora/oscyloskopu.

---

2) Funkcjonalności jako komendy (jedno CLI, wiele modułów)

2.1 GPIO: przycisk i LED

Komendy (przykładowe):

- `gpio status` – pokaż stan przycisku, stan LED
- `led on|off|toggle`
- `button mode poll|irq` – przełącz obsługę przycisku (polling vs EXTI)

W tle:

- LED mruga zgodnie z wcześniejszymi zaleceniami ●
  Przycisk:
  - Jedno przyciśnięcie zwiększa częstotliwość mrugania pojedynczego
  - Podwójne naciśnięcie zmniejsza częstotliwość
  - Potrójne wciśnięcie zapisuje aktualne ustawienia jako bazowe w pamięci Flash
  - Dłuższe przyciśnięcie (około 5 sekund), reset do ustawień bazowych

---

2.2 PWM do LED

Po wysłaniu odpowiedniej komendy włączającej tryb PWM miganie diodą jest zawieszane i działamy sygnałem PWM. Diagnostyka na diodzie LED wciąż ma działać i jest funkcją nadrzędna.

Komendy (przykładowe):

- `pwm on|off` – automatyczna rampa jasności (task w schedulerze)
- `pwm set <0-100>` – ustaw wypełnienie
- `pwm freq <Hz>` – jeśli chcesz zmienic czestotliwosc
- `pwm toggle` – powoduje cykliczne ściemnianie i rozjaśnianie diody LED

W tle:

- PWM generowane timerem

2.3 DMA

DMA „ma mieć sens”, więc robimy to tak:

- **UART RX DMA** (ciągły odbiór)
- **UART TX DMA** (logi i odpowiedzi) **Komendy diagnostyczne:**

- `dma stat` – pokaż wskaźniki bufora RX, ilość odebranych bajtów, overruns
- `uart flood <n>` – wyślij dużą paczkę danych (test TX DMA i terminala)

2.4 Flash: zapis i odczyt (prawdziwa trwałość)

Komendy:

- `flash write <addr> <hex>` – zapisz słowo/rekord do wydzielonej strony
- `flash read <addr> <len>` - odczytaj
- `flash erase` – kasuj stronę testową

W tle:

- Prosty „rekord” z CRC i numerem wersji
- Na starcie firmware czyta rekord i raportuje stan (OK/CRC FAIL/EMPTY)

---

3) Podsumowanie komend

GPIO + LED + przycisk

1. gpio status
    a. stan przycisku
    b. stan LED
    c. tryb obsługi przycisku (poll / irq)
2. led on

        a. Włącza LED (tryb manualny)
    3. led off
        a. Wyłącza LED
    4. led toggle
        a. Zmienia stan LED
    5. button mode poll
        a. Obsługa przycisku przez polling
    6. button mode irq
        a. Obsługa przycisku przez EXTI

PWM

1. pwm on
    a. Włącza tryb PWM (zawiesza miganie zadaniowe)
2. pwm off
    a. Wyłącza PWM (powrót do trybu mrugania)
3. pwm set <0-100>
    a. Ustawienie wypełnienia
4. pwm freq <Hz>
    a. Zmiana częstotliwości
5. pwm toggle
    a. Cykliczne ściemnianie/rozjaśnianie (rampa w schedulerze)

DMA / UART

1. dma stat
    a. wskaźniki bufora RX
    b. ilość odebranych bajtów
    c. ilość overruns
2. uart flood <n>
    a. Wysyła dużą paczkę danych (test TX DMA

Flash

1. flash write <addr> <hex>
    a. Zapis do wydzielonej strony
2. flash read <addr> <len>
    a. Odczyt
3. flash erase
    a. Kasowanie strony testowej

Błędy parsera (wymagane w projekcie)

Muszą być zaimplementowane (z indywidualnym wzorem mrugnięć LED):

1. niepoprawny początek komunikatu
2. błędna druga część komendy
3. nieznana komenda
4. niekompletna komenda
5. inne błędy składni / parametrów

---

4) Raport projektu

4.1 Opis struktury programu

W raporcie ma być opisana struktura projektu, to jak zostały ułożone funkcje w funkcji main() oraz jak są uruchamiane pozostałe funkcjonalności.

4.2 Dokumentacja komend UART

Należy opisać jak się komunikować z urządzeniem i jak składać komendy. Dodatkowo opis dostępnych komend.

4.3 Opis funkcjonalności

Dokładny opis działających funkcji oraz sposobu ich implementacji.

4.4 Debugownaie i testy

Każda funkcjonalność ma być przetestowana dostępnymi metodami oraz zmierzony czas wykonania przy pomocy analizatora stanów logicznych. Opisać różnice między metodami debugowania, ich zalety i wady.